NewFUHI
システム取扱説明書

予約・店舗・IoT・決済・在庫・給与 統合管理プラットフォーム

Version 1.0 — 2026年3月

Confidential — 社内用資料

目次

1. システム概要
2. システム構成
3. データベースモデル一覧
4. 画面一覧・URLマップ
5. APIリファレンス
6. 管理画面の操作手順
7. 予約フロー
8. 決済連携
9. QRテーブル注文
10. IoTデバイス管理
11. 物件管理・監視
12. シフト・勤怠・給与管理
13. 在庫管理
14. ECショップ
15. セキュリティ機能
16. AWSコスト監視
17. 定期タスク（Celery）
18. 管理コマンドリファレンス
19. デプロイ手順
20. 多言語対応
21. トラブルシューティング

1. システム概要

1.1 NewFUHI とは

NewFUHI は、占いサロン・飲食店・宿泊施設などの店舗運営を統合的に管理する Django ベースの Web プラットフォームです。予約管理、決済、IoT デバイス制御、在庫管理、シフト・給与管理、EC ショップ、物件監視など幅広い業務をカバーします。

1.2 技術スタック

レイヤー	技術
バックエンド	Django / Django REST Framework / Python 3
タスクキュー	Celery + Redis
データベース	SQLite（開発）/ PostgreSQL・RDS（本番）
認証	Django Auth + LINE OAuth2（python-social-auth）
暗号化	cryptography（Fernet）
QRコード	qrcode[pil]
AWS連携	boto3
Web サーバー	Nginx + Gunicorn
CI/CD	GitHub Actions
プロセス管理	systemd

1.3 対応言語（7言語）

コード	言語	備考
ja	日本語	デフォルト言語
en	English	
zh-hant	繁體中文	台湾・香港向け
zh-hans	简体中文	中国大陸向け
ko	한국어	
es	Español	
pt	Português	

2. システム構成

2.1 ディレクトリ構造

```
NewFUHI/
├── manage.py
├── project/                    # Django プロジェクト設定
│   ├── urls.py                # メイン URL ルーティング
│   └── settings/
│       ├── base.py            # 共通設定
│       ├── local.py           # 開発用設定
│       ├── production.py      # 本番設定
│       └── staging.py         # ステージング設定
├── booking/                    # メインアプリ
│   ├── models.py              # 全データベースモデル（1638行）
│   ├── views.py               # メインビュー（2806行）
│   ├── views_debug.py         # デバッグパネル
│   ├── views_dashboard.py     # センサーダッシュボード
│   ├── views_restaurant_dashboard.py  # 売上ダッシュボード
│   ├── views_property.py      # 物件管理ビュー
│   ├── views_chat.py          # AIチャットビュー
│   ├── urls.py                # ページ URL（151行）
│   ├── api_urls.py            # API エンドポイント（87行）
│   ├── table_urls.py          # テーブル注文 URL
│   ├── admin.py               # 管理画面カスタマイズ（1215行）
│   ├── admin_site.py          # ロールベース管理サイト
│   ├── forms.py               # フォーム定義
│   ├── tasks.py               # Celery タスク
│   ├── middleware.py          # セキュリティ監査ミドルウェア
│   ├── management/commands/   # 管理コマンド（5個）
│   └── templates/booking/     # テンプレート（30+）
├── config/                     # デプロイ設定
│   ├── nginx/                 # Nginx設定
│   ├── gunicorn/              # Gunicorn設定
│   ├── systemd/               # サービス定義
│   └── logrotate/             # ログローテーション
├── locale/                     # 翻訳ファイル（7言語）
├── static/                     # 静的ファイル
├── media/                      # アップロードファイル
├── celery_config.py           # Celery設定
└── requirements.txt           # 依存パッケージ
```

2.2 設定ファイル

ファイル	内容
`project/settings/base.py`	共通設定（INSTALLED_APPS, MIDDLEWARE, AUTH, i18n）
`project/settings/local.py`	開発環境（DEBUG=True, SQLite, ローカルメール）
`project/settings/production.py`	本番環境（SSL, HSTS, セキュアクッキー）
`project/settings/staging.py`	ステージング環境
`celery_config.py`	Celery Beat スケジュール定義

2.3 環境変数一覧

必須

変数名	説明
`SECRET_KEY`	Django シークレットキー
`LINE_CHANNEL_ID`	LINE Bot チャネル ID
`LINE_CHANNEL_SECRET`	LINE Bot チャネルシークレット
`LINE_REDIRECT_URL`	LINE OAuth リダイレクト URL
`LINE_ACCESS_TOKEN`	LINE アクセストークン
`PAYMENT_API_KEY`	決済 API キー
`PAYMENT_API_URL`	決済 API URL
`WEBHOOK_URL_BASE`	Webhook ベース URL
`CANCEL_URL`	キャンセル URL
`LINE_USER_ID_ENCRYPTION_KEY`	LINE ID Fernet 暗号化キー
`LINE_USER_ID_HASH_PEPPER`	LINE ID ハッシュペッパー
`IOT_ENCRYPTION_KEY`	IoT 資格情報暗号化キー

オプション

変数名	説明	デフォルト
`DEBUG`	デバッグモード	True（開発）
`ALLOWED_HOSTS`	許可ホスト	localhost,127.0.0.1
`DB_ENGINE`	DB エンジン	sqlite3
`DB_NAME`	DB 名	db.sqlite3
`DATABASE_URL`	DB 接続 URL	—
`CELERY_BROKER_URL`	Redis ブローカー	redis://localhost:6379/0
`EMAIL_HOST`	SMTP ホスト	—
`GEMINI_API_KEY`	Google Gemini API	—
`LOG_LEVEL`	ログレベル	DEBUG

2.4 INSTALLED_APPS

アプリ	用途
`booking`	メインアプリ（全ビジネスロジック）
`rest_framework`	REST API フレームワーク
`social_django`	LINE OAuth2 ソーシャル認証
`booking.custom_auth`	カスタム認証（管理画面表示名変更）

2.5 ミドルウェア

#	ミドルウェア	役割
1	SecurityMiddleware	HTTPS リダイレクト、HSTS
2	SessionMiddleware	セッション管理
3	LocaleMiddleware	多言語切替
4	CommonMiddleware	URL 正規化
5	CsrfViewMiddleware	CSRF 保護
6	AuthenticationMiddleware	ユーザー認証
7	MessageMiddleware	メッセージフレームワーク
8	XFrameOptionsMiddleware	クリックジャッキング防止
9	SecurityAuditMiddleware	セキュリティイベント記録

3. データベースモデル一覧

全モデルは `booking/models.py`（1638行）に定義されています。

3.1 認証・ユーザー管理

Timer

フィールド	型	説明
user_id	CharField(255, unique)	ユーザー識別子
start_time	DateTimeField(auto_now_add)	開始時刻
end_time	DateTimeField(null)	終了時刻

3.2 店舗管理

Store（店舗）

フィールド	型	説明
name	CharField(255)	店舗名
thumbnail	ImageField	サムネイル画像
address	CharField(255)	住所
business_hours	CharField(255)	営業時間
nearest_station	CharField(255)	最寄り駅
regular_holiday	CharField(255)	定休日
description	TextField	説明文
is_recommended	BooleanField	おすすめフラグ
default_language	CharField(10)	デフォルト言語

Staff（スタッフ）

フィールド	型	説明
name	CharField(50)	名前
user	OneToOneField → User	Django ユーザー
store	ForeignKey → Store	所属店舗
line_id	CharField(50, null)	LINE ID
staff_type	CharField(choices)	fortune_teller / store_staff
is_recommended	BooleanField	おすすめフラグ
is_store_manager	BooleanField	店長フラグ
is_owner	BooleanField	オーナーフラグ
is_developer	BooleanField	開発者フラグ
thumbnail	ImageField	プロフィール画像
introduction	TextField	自己紹介
price	IntegerField	料金

StoreScheduleConfig（営業時間設定）

フィールド	型	説明
store	OneToOneField → Store	対象店舗
open_hour	IntegerField(0-23)	開店時刻
close_hour	IntegerField(0-23)	閉店時刻
slot_duration	IntegerField	予約枠（15/30/45/60分）

3.3 予約管理

Schedule（予約）

フィールド	型	説明
reservation_number	CharField(255, unique)	予約番号（UUID）
start	DateTimeField	開始日時
end	DateTimeField	終了日時
staff	ForeignKey → Staff	担当スタッフ
customer_name	CharField(255, null)	顧客名
hashed_id	CharField(255, null)	ハッシュ済 ID
line_user_hash	CharField(64, null)	LINE ID ハッシュ
line_user_enc	TextField(null)	LINE ID 暗号化
is_temporary	BooleanField	仮予約フラグ
is_cancelled	BooleanField	キャンセルフラグ
price	IntegerField	料金
memo	TextField	メモ
temporary_booked_at	DateTimeField(null)	仮予約日時
booking_channel	CharField(choices)	line / email
customer_email	EmailField(null)	メールアドレス
email_otp_hash	CharField(64, null)	OTP ハッシュ
email_otp_expires	DateTimeField(null)	OTP 有効期限
email_verified	BooleanField	メール認証済
payment_url	URLField(null)	決済 URL
checkin_qr	ImageField	チェックイン QR
is_checked_in	BooleanField	チェックイン済
checked_in_at	DateTimeField(null)	チェックイン日時

3.4 IoT デバイス

IoTDevice

フィールド	型	説明
name	CharField(100)	デバイス名
store	ForeignKey → Store	設置店舗
device_type	CharField(choices)	multi / door / other
external_id	CharField(255, unique)	AWS デバイス ID
api_key_hash	CharField(64)	API キー SHA-256 ハッシュ
api_key_prefix	CharField(8)	表示用プレフィックス
mq9_threshold	FloatField(null)	MQ-9 ガスセンサー閾値
alert_enabled	BooleanField	アラート有効
alert_email	EmailField	アラート送信先メール
alert_line_user_id	CharField(255)	アラート送信先 LINE
wifi_ssid	CharField(64)	Wi-Fi SSID
wifi_password_enc	CharField(512)	Wi-Fi パスワード（暗号化）
pending_ir_command	TextField	保留中 IR コマンド（JSON）
is_active	BooleanField	有効フラグ
last_seen_at	DateTimeField(null)	最終通信日時

IoTEvent（センサーイベント）

フィールド	型	説明
device	ForeignKey → IoTDevice	デバイス
created_at	DateTimeField(auto)	記録日時
event_type	CharField(50)	イベント種別
payload	TextField	JSON ペイロード
mq9_value	FloatField(null)	ガスセンサー値
light_value	FloatField(null)	照度センサー値
sound_value	FloatField(null)	音センサー値
pir_triggered	BooleanField(null)	人感センサー検知

IRCode（赤外線リモコンコード）

フィールド	型	説明
device	ForeignKey → IoTDevice	デバイス
name	CharField(100)	コード名
protocol	CharField(20)	プロトコル（NEC等）
code	CharField(20)	IRコード値
raw_data	TextField	生データ

3.5 商品・注文・在庫

Category（カテゴリ）

フィールド	型	説明
store	ForeignKey → Store	店舗
name	CharField(100)	カテゴリ名
sort_order	IntegerField	表示順

Product（商品）

フィールド	型	説明
store	ForeignKey → Store	店舗
category	ForeignKey → Category(null)	カテゴリ
sku	CharField(64)	商品コード
name	CharField(200)	商品名
description	TextField	商品説明
price	IntegerField	価格
stock	IntegerField	在庫数
low_stock_threshold	IntegerField	低在庫閾値
is_active	BooleanField	有効フラグ
is_ec_visible	BooleanField	EC ショップ表示
image	ImageField	商品画像
popularity	IntegerField	人気スコア
margin_rate	FloatField	利益率

ProductTranslation（商品翻訳）

フィールド	型	説明
product	ForeignKey → Product	商品
lang	CharField(10)	言語コード
name	CharField(200)	翻訳名
description	TextField	翻訳説明

> unique_together: (product, lang) — 1商品につき各言語1翻訳

Order（注文）

フィールド	型	説明
store	ForeignKey → Store	店舗
schedule	ForeignKey → Schedule(null)	関連予約
table_label	CharField(50)	テーブル番号
table_seat	ForeignKey → TableSeat(null)	テーブル席
status	CharField(choices)	OPEN / CLOSED
created_at	DateTimeField(auto)	作成日時

OrderItem（注文明細）

フィールド	型	説明
order	ForeignKey → Order	注文
product	ForeignKey → Product	商品
qty	IntegerField	数量
unit_price	IntegerField	単価
status	CharField(choices)	ORDERED → PREPARING → SERVED → CLOSED
note	CharField(255)	備考

StockMovement（在庫移動）

フィールド	型	説明
store	ForeignKey → Store	店舗
product	ForeignKey → Product	商品
movement_type	CharField(choices)	IN / OUT / ADJUST
qty	IntegerField	数量
by_staff	ForeignKey → Staff(null)	操作者
note	CharField(255)	備考

3.6 物件管理

Property（物件）

フィールド	型	説明
name	CharField(200)	物件名
address	CharField(300)	住所
property_type	CharField(choices)	apartment / house / office / store
owner_name	CharField(100)	オーナー名
owner_contact	CharField(200)	オーナー連絡先
store	ForeignKey → Store(null)	関連店舗
is_active	BooleanField	有効フラグ

PropertyDevice / PropertyAlert

PropertyDevice: 物件とIoTデバイスの紐付け（location_label で設置場所を記録）。
PropertyAlert: ガス漏れ（gas_leak）、動体未検知（no_motion）、デバイスオフライン（device_offline）のアラート。severity は critical / warning / info。

3.7 シフト管理

ShiftPeriod（シフト期間）

フィールド	型	説明
store	ForeignKey → Store	店舗
year_month	DateField	対象月
deadline	DateTimeField	申請締切
status	CharField(choices)	open → closed → scheduled → approved

ShiftRequest / ShiftAssignment

ShiftRequest: スタッフの希望シフト（preference: available / preferred / unavailable）。
ShiftAssignment: 確定シフト（is_synced で Schedule への同期状態を管理）。

3.8 テーブル・決済

TableSeat（テーブル席）

フィールド	型	説明
id	UUIDField(PK)	UUID 主キー
store	ForeignKey → Store	店舗
label	CharField(50)	テーブル名（例: A1）
is_active	BooleanField	有効フラグ
qr_code	ImageField	QR コード画像

PaymentMethod（決済方法）

フィールド	型	説明
store	ForeignKey → Store	店舗
method_type	CharField(choices)	cash / coiney / paypay / ic / other
display_name	CharField(100)	表示名
is_enabled	BooleanField	有効フラグ
api_key / api_secret	CharField	API 認証情報
extra_config	JSONField	追加設定

3.9 給与管理

EmploymentContract（雇用契約）

フィールド	型	説明
staff	OneToOneField → Staff	スタッフ
employment_type	CharField	full_time / part_time / contract
pay_type	CharField	hourly / monthly
hourly_rate	IntegerField	時給
monthly_salary	IntegerField	月給
通勤・住宅・家族手当	IntegerField × 3	各種手当
standard_monthly_remuneration	IntegerField	標準報酬月額（社保基準）
resident_tax_monthly	IntegerField	住民税月額
birth_date	DateField(null)	生年月日（介護保険判定用）

PayrollPeriod / PayrollEntry / PayrollDeduction

PayrollPeriod: 給与計算期間（status: draft → calculating → confirmed → paid）。
PayrollEntry: 個人別給与明細（労働時間・基本給・残業代・控除・手取り）。
PayrollDeduction: 控除項目（所得税・住民税・厚生年金・健保・雇用保険・介護保険・労災）。

SalaryStructure（給与計算設定）

フィールド	型	説明
pension_rate	Decimal(5,3)	厚生年金率 %（デフォルト 9.150）
health_insurance_rate	Decimal(5,3)	健康保険率 %（デフォルト 5.000）
employment_insurance_rate	Decimal(5,3)	雇用保険率 %（デフォルト 0.600）
long_term_care_rate	Decimal(5,3)	介護保険率 %（40歳以上, デフォルト 0.820）
workers_comp_rate	Decimal(5,3)	労災保険率 %（デフォルト 0.300）
overtime_multiplier	Decimal(4,2)	残業倍率（デフォルト 1.25）
late_night_multiplier	Decimal(4,2)	深夜倍率（デフォルト 1.35）
holiday_multiplier	Decimal(4,2)	休日倍率（デフォルト 1.50）

3.10 CMS・ホームページ

SiteSettings（サイト設定・シングルトン）

サイト名、表示セクション制御（カード・サイドバー・ランキング等のON/OFF）、SNS連携URL、スタッフ呼称（デフォルト：キャスト）、プライバシーポリシー・特商法HTMLなど。

HomepageCustomBlock / HeroBanner / BannerAd / ExternalLink

ホームページのカスタムHTMLブロック、ヒーローバナー（画像+リンク先設定）、バナー広告（5つの配置位置）、外部リンク。いずれも sort_order / is_active で表示制御。

3.11 管理画面カスタマイズ

AdminTheme / DashboardLayout / AdminMenuConfig

AdminTheme: 店舗ごとの管理画面テーマカラー。DashboardLayout: ユーザーごとのダッシュボードレイアウト（JSON）+ダークモード。AdminMenuConfig: ロール別管理画面メニュー許可設定。

3.12 セキュリティ・監視

SecurityAudit / SecurityLog / CostReport / SystemConfig

SecurityAudit: セキュリティ監査結果（category: django_settings / credentials / endpoints / infrastructure / dependencies）。SecurityLog: ログイン・認証失敗・レート制限イベント。CostReport: AWS コストレポート。SystemConfig: Key-Value 設定ストア（シングルトンパターン）。

3.13 会社情報・お知らせ

Company / Notice / Media

Company: 会社名・住所・電話番号。Notice: お知らせ（タイトル・リンク・本文）。Media: メディア掲載情報（URLからサムネイル自動取得、内部ネットワークアクセス防止セキュリティチェック付き）。

4. 画面一覧・URLマップ

4.1 顧客向けページ（booking/urls.py）

URL	ビュー	name	説明
`/`	BookingTopPage	booking_top	トップページ
`/stores/`	StoreList	store_list	店舗一覧
`/fortune-tellers/`	AllFortuneTellerList	all_fortune_tellers	キャスト一覧
`/date-calendar/`	DateFirstCalendar	date_first_calendar	日付優先カレンダー
`/store/<id>/staffs/`	StaffList	staff_list	店舗スタッフ一覧
`/staff/<id>/calendar/`	StaffCalendar	staff_calendar	スタッフカレンダー
`/staff/<id>/prebooking/...`	PreBooking	prebooking	予約フォーム
`/booking/channel-choice/`	BookingChannelChoice	channel_choice	LINE/メール選択
`/booking/email/`	EmailBookingView	email_booking	メール予約フォーム
`/booking/email/verify/`	EmailVerifyView	email_verify	メール OTP 認証
`/line_enter/`	LineEnterView	line_enter	LINE ログイン開始
`/booking/login/line/success/`	LineCallbackView	line_success	LINE OAuth コールバック
`/cancel_reservation/<id>/`	CancelReservationView	cancel_reservation	予約キャンセル
`/reservation/<no>/qr/`	ReservationQRView	reservation_qr	予約 QR コード
`/checkin/`	CheckinScanView	checkin_scan	QR チェックインスキャン
`/privacy/`	PrivacyPolicyView	privacy_policy	プライバシーポリシー
`/tokushoho/`	TokushohoView	tokushoho	特定商取引法に基づく表記
`/help/`	HelpView	help	ヘルプページ

4.2 スタッフ向けページ

URL	ビュー	name	説明
`/mypage/`	MyPage	my_page	マイページ
`/mypage/<id>/`	MyPageWithPk	my_page_with_pk	マイページ（ID指定）
`/mypage/<id>/calendar/`	MyPageCalendar	my_page_calendar	スケジュールカレンダー
`/mypage/<id>/config/.../`	MyPageDayDetail	my_page_day_detail	日別設定
`/mypage/schedule/<id>/`	MyPageSchedule	my_page_schedule	予約編集
`/shift/`	StaffShiftCalendarView	staff_shift_calendar	シフトカレンダー
`/shift/<id>/submit/`	StaffShiftSubmitView	staff_shift_submit	シフト希望提出

4.3 飲食・注文ページ

URL	ビュー	name	説明
`/menu/<store_id>/`	CustomerMenuView	customer_menu	メニュー一覧
`/stock/inbound/`	InboundQRView	inbound_qr	QR 入庫
`/shop/`	ShopView	shop	EC ショップ
`/shop/cart/`	CartView	shop_cart	カート
`/shop/checkout/`	ShopCheckoutView	shop_checkout	チェックアウト

4.4 テーブル注文（table_urls.py）

URL	ビュー	name	説明
`/table/<uuid>/`	TableMenuView	table_menu	テーブルメニュー
`/table/<uuid>/cart/`	TableCartView	table_cart	テーブルカート
`/table/<uuid>/order/`	TableOrderView	table_order	テーブル注文確認
`/table/<uuid>/history/`	TableOrderHistoryView	table_history	注文履歴
`/table/<uuid>/checkout/`	TableCheckoutView	table_checkout	テーブル会計

4.5 IoT・物件ページ

URL	ビュー	name	説明
`/dashboard/sensors/`	IoTSensorDashboardView	iot_sensor_dashboard	センサーダッシュボード
`/properties/`	PropertyListView	property_list	物件一覧
`/properties/<id>/`	PropertyDetailView	property_detail	物件詳細

5. APIリファレンス

全 API エンドポイントは `booking/api_urls.py` で定義されています（prefix: `/api/`）。

5.1 IoT API

メソッド	エンドポイント	認証	説明
POST	`/api/iot/events/`	API Key (X-API-Key ヘッダー)	センサーデータ送信
GET	`/api/iot/config/`	API Key	デバイス設定取得（pending_ir_command 含む）
GET	`/api/iot/sensors/data/`	ログイン	センサーデータ取得（グラフ用）
GET	`/api/iot/sensors/pir-events/`	ログイン	人感センサーイベント
GET	`/api/iot/sensors/pir-status/`	ログイン	人感センサー状態
POST	`/api/iot/ir/send/`	ログイン	IR コマンド送信

5.2 注文 API

メソッド	エンドポイント	説明
GET	`/api/menu`	メニュー JSON 取得（store_id パラメータ）
GET	`/api/products/alternatives/`	代替商品提案
POST	`/api/orders/create/`	注文作成
GET	`/api/orders/status/`	注文ステータス取得
POST	`/api/staff/orders/served/`	提供済みマーク
PATCH	`/api/staff/orders/items/<id>/status/`	注文品ステータス更新

5.3 テーブル注文 API

メソッド	エンドポイント	説明
POST	`/api/table/<uuid>/cart/add/`	テーブルカートに追加
POST	`/api/table/<uuid>/cart/update/`	テーブルカート更新
POST	`/api/table/<uuid>/cart/remove/`	テーブルカートから削除
POST	`/api/table/<uuid>/order/create/`	テーブル注文作成
GET	`/api/table/<uuid>/orders/status/`	テーブル注文ステータス

5.4 在庫 API

メソッド	エンドポイント	説明
POST	`/api/stock/inbound/apply/`	入庫処理適用

5.5 EC ショップ API

メソッド	エンドポイント	説明
POST	`/api/shop/cart/add/`	カートに追加
POST	`/api/shop/cart/update/`	カート更新
POST	`/api/shop/cart/remove/`	カートから削除

5.6 ダッシュボード API

メソッド	エンドポイント	説明
GET/POST	`/api/dashboard/layout/`	ダッシュボードレイアウト取得・保存
GET	`/api/dashboard/reservations/`	予約統計
GET	`/api/dashboard/sales/`	売上統計
GET	`/api/dashboard/staff-performance/`	スタッフ実績
GET	`/api/dashboard/shift-summary/`	シフトサマリー

5.7 物件 API

メソッド	エンドポイント	説明
GET	`/api/properties/<id>/status/`	物件ステータス
POST	`/api/alerts/<id>/resolve/`	アラート解決

5.8 チェックイン API

メソッド	エンドポイント	説明
POST	`/api/checkin/`	QR チェックイン処理

5.9 AI チャット API

メソッド	エンドポイント	説明
POST	`/api/chat/admin/`	管理者向け AI チャット
POST	`/api/chat/guide/`	ガイド AI チャット

5.10 デバッグ API

メソッド	エンドポイント	説明
GET	`/api/debug/panel/`	デバッグパネルデータ
POST	`/api/debug/log-level/`	ログレベル動的変更

5.11 予約時刻 API

メソッド	エンドポイント	説明
GET	`/api/endTime`	終了時刻 JSON
GET	`/api/currentTime`	現在時刻 JSON
GET	`/api/reservation/<id>/`	予約時刻取得
GET	`/api/reservation_times/<id>/`	予約時刻一覧

6. 管理画面の操作手順

6.1 アクセス方法

管理画面は `/admin/` でアクセスします。カスタム管理サイト `RoleBasedAdminSite` を使用しており、ユーザーのロールに応じてメニューが動的にフィルタリングされます。

6.2 ロールベースアクセス制御

ロール	判定条件	権限範囲
superuser	User.is_superuser = True	全モデル表示（Django 標準権限）
developer	Staff.is_developer = True	全モデル表示
owner	Staff.is_owner = True	全モデル表示
manager	Staff.is_store_manager = True	約30モデル（予約・注文・商品・シフト・給与・IoT・物件・CMS等）
staff	上記いずれでもない Staff	7モデル（予約・注文・スタッフ・IoT・商品・シフト申請）

6.3 GROUP_MAP（メニューグループ定義）

管理画面のメニューは以下の15グループに整理されて表示されます：

グループ名	含まれるモデル
予約管理	Schedule
シフト管理	ShiftPeriod, ShiftRequest, ShiftAssignment
キャスト管理	Staff, Store, StoreScheduleConfig
給与管理	PayrollPeriod, PayrollEntry, EmploymentContract, SalaryStructure
勤怠管理	WorkAttendance
在庫管理	Category, Product
注文管理	Order
テーブル注文	TableSeat, PaymentMethod
IoT管理	IoTDevice
物件管理	Property
予約ページ情報	Company, Notice, Media
ページ設定	SiteSettings, HomepageCustomBlock, HeroBanner, BannerAd, ExternalLink
システム	SystemConfig, AdminTheme, DashboardLayout, AdminMenuConfig
セキュリティ	SecurityAudit, SecurityLog, CostReport
ユーザーアカウント管理	User, Group

6.4 メニュー設定のカスタマイズ

`AdminMenuConfig` モデルでロールごとの表示メニューを DB から動的に変更可能です。5分間のキャッシュ TTL あり。DB に設定がない場合は `DEFAULT_ALLOWED_MODELS` にフォールバックします。

7. 予約フロー

7.1 LINE 予約フロー

1. 顧客がトップページで店舗 → スタッフ → 日時を選択
2. `/booking/channel-choice/` で LINE を選択
3. `/line_enter/` → LINE OAuth2 認証開始
4. LINE ログイン成功 → `/booking/login/line/success/` コールバック
5. 仮予約（Schedule）作成（`is_temporary=True`）
6. LINE User ID を暗号化（Fernet）＋ハッシュ（SHA-256 + pepper）して保存
7. 決済 URL（Coiney）を生成し `payment_url` に保存
8. 顧客が決済完了 → Webhook で `is_temporary=False` に更新
9. チェックイン QR コード生成・保存

> 仮予約は Celery タスク（毎分実行）で 10分超過分を自動削除します。

7.2 メール予約フロー

1. 顧客が `/booking/channel-choice/` で「メール」を選択
2. `/booking/email/` でメールアドレス入力
3. OTP（ワンタイムパスワード）をメール送信
4. `/booking/email/verify/` で OTP 入力・認証
5. 認証成功 → 仮予約作成（`booking_channel='email'`, `email_verified=True`）
6. 以降は LINE フローと同じ（決済→確定→QR生成）

7.3 チェックイン

1. 店舗スタッフが `/checkin/` で QR スキャナーを起動
2. 顧客のチェックイン QR コードを読み取り
3. `/api/checkin/` API でチェックイン処理
4. `is_checked_in=True`, `checked_in_at` 記録

7.4 予約キャンセル

`/cancel_reservation/<schedule_id>/` で予約をキャンセル。`is_cancelled=True` に更新。

8. 決済連携

8.1 Coiney 決済フロー

1. 仮予約作成時に決済 API（`PAYMENT_API_URL`）に決済リクエスト送信
2. `PAYMENT_API_KEY` で API 認証
3. 決済 URL を取得 → Schedule の `payment_url` に保存
4. 顧客を決済ページにリダイレクト
5. 決済完了 → Coiney が Webhook を送信

8.2 Webhook 処理

```
# URL: /api/coiney/webhook/<orderId>/
# ビュー: coiney_webhook(request, orderId)
# 処理:
#   1. orderId で Schedule を検索
#   2. is_temporary = False に更新（確定）
#   3. チェックイン QR コード生成
#   4. LINE で確認メッセージ送信
```

8.3 PaymentMethod モデル

店舗ごとに複数の決済方法を設定可能：cash / coiney / paypay / ic / other。`api_key`, `api_secret`, `extra_config`（JSON）で各決済プロバイダーの設定を管理。

9. QRテーブル注文

9.1 概要

テーブルに設置した QR コードをスマートフォンで読み取り、メニュー閲覧→カート→注文→会計をセルフサービスで完結するシステム。

9.2 フロー

1. **QR 読み取り** → `/table/<uuid>/` テーブルメニュー表示
2. **商品選択** → `/api/table/<uuid>/cart/add/` でカート追加
3. **カート確認** → `/table/<uuid>/cart/`
4. **注文確定** → `/api/table/<uuid>/order/create/`
5. **注文状況** → `/api/table/<uuid>/orders/status/` でリアルタイム確認
6. **注文履歴** → `/table/<uuid>/history/`
7. **会計** → `/table/<uuid>/checkout/`

9.3 注文ステータス遷移

```
ORDERED → PREPARING → SERVED → CLOSED
 注文済    調理中     提供済   会計済
```

スタッフは `/api/staff/orders/items/<id>/status/` で各アイテムのステータスを更新。

9.4 テーブル管理

管理画面の「テーブル注文」グループから `TableSeat` を登録。UUID ベースの QR コード画像が自動生成されます。

10. IoTデバイス管理

10.1 デバイス登録

管理画面の「IoT管理」から `IoTDevice` を登録。デバイスタイプ：multi（マルチセンサー）/ door（ドアセンサー）/ other。

10.2 API 認証

デバイスは `X-API-Key` ヘッダーで認証。API キーは SHA-256 ハッシュで保存（平文は保存しない）。登録時にのみ平文表示。

10.3 センサーデータ送信

```
POST /api/iot/events/
Headers: X-API-Key: <api_key>
Body: {
  "event_type": "sensor_reading",
  "mq9_value": 150.5,    // ガスセンサー値
  "light_value": 450.0,  // 照度値
  "sound_value": 35.2,   // 音量値
  "pir_triggered": false  // 人感センサー
}
```

10.4 ガスアラート

`mq9_value` が `mq9_threshold` を超えた場合、Celery タスク `trigger_gas_alert` が発火。メール＋LINE で緊急通知。

10.5 IR スマートハブ

1. 管理画面で `IRCode`（学習済み赤外線コード）を登録
2. `POST /api/iot/ir/send/` でコマンド送信
3. デバイスが `GET /api/iot/config/` で `pending_ir_command` を取得して実行

10.6 Wi-Fi パスワード管理

デバイスの Wi-Fi パスワードは Fernet 暗号化して保存。`set_wifi_password()` / `get_wifi_password()` メソッドで暗号化・復号。

10.7 センサーダッシュボード

`/dashboard/sensors/` でリアルタイムセンサーデータを可視化。API: `/api/iot/sensors/data/`, `/api/iot/sensors/pir-events/`, `/api/iot/sensors/pir-status/`。

11. 物件管理・監視

11.1 物件登録

管理画面「物件管理」から `Property` を登録。property_type: apartment / house / office / store。IoTDevice を `PropertyDevice` で紐付け、設置場所（location_label）を記録。

11.2 アラート機能

アラートタイプ	トリガー条件	severity
gas_leak	MQ-9 値が閾値を超過	critical
no_motion	一定期間 PIR 検知なし	warning
device_offline	デバイスが一定期間通信なし	warning
custom	カスタム条件	設定による

11.3 定期チェック

Celery Beat で5分間隔で `check_property_alerts` タスクを実行。アラート発生時は `PropertyAlert` レコードを作成。

11.4 アラート解決

`POST /api/alerts/<id>/resolve/` で `is_resolved=True`, `resolved_at` を記録。

12. シフト・勤怠・給与管理

12.1 シフト管理フロー

1. **シフト期間作成**：管理画面で `ShiftPeriod` 作成（対象月・締切設定、status=open）
2. **シフト希望提出**：スタッフが `/shift/<period_id>/submit/` で希望提出（available / preferred / unavailable）
3. **シフト確定**：管理者が `ShiftAssignment` を作成（日付・開始時刻・終了時刻）
4. **Schedule 同期**：`is_synced` フラグで予約カレンダーへの同期状態を管理

12.2 勤怠管理

`WorkAttendance` で日次の勤怠記録。シフトからの自動生成（source=shift）、手動入力（manual）、修正（corrected）の3ソース。

フィールド	説明
regular_minutes	通常勤務時間（分）
overtime_minutes	残業時間（分）
late_night_minutes	深夜勤務時間（分）
holiday_minutes	休日勤務時間（分）
break_minutes	休憩時間（分）

12.3 給与計算

1. `PayrollPeriod` 作成（status=draft）
2. 計算実行（status=calculating）→ `PayrollEntry` 自動生成
3. `EmploymentContract` に基づき基本給計算（時給制 or 月給制）
4. `SalaryStructure` の倍率で残業代・深夜手当・休日手当を計算
5. `PayrollDeduction` で社会保険料・税金を自動控除
6. 確定（status=confirmed）→ 支給（status=paid）

12.4 社会保険控除項目

項目	デフォルト率	備考
厚生年金	9.150%	標準報酬月額ベース
健康保険	5.000%	標準報酬月額ベース
雇用保険	0.600%	総支給額ベース
介護保険	0.820%	40歳以上のみ（birth_date で判定）
労災保険	0.300%	事業主負担（is_employer_only）
所得税	—	源泉徴収税額表に基づく
住民税	—	resident_tax_monthly（定額）

13. 在庫管理

13.1 在庫の基本操作

操作	movement_type	stock への影響
入庫	IN	+ qty
出庫	OUT	- qty
棚卸調整	ADJUST	指定数量に設定

全ての在庫変動は `StockMovement` に記録され、トレーサビリティを確保。

13.2 低在庫通知

Celery Beat タスク `check_low_stock_and_notify`（毎日09:00）で `stock ≤ low_stock_threshold` の商品を検出し、LINE で通知。`last_low_stock_notified_at` で重複通知を防止。

13.3 QR 入庫

`/stock/inbound/` ページで商品 QR をスキャンし、`/api/stock/inbound/apply/` API で入庫処理。

13.4 注文による自動出庫

注文作成時に `apply_stock_movement(product, 'OUT', qty)` が呼ばれ、在庫を自動減算。
`allow_negative=False` で在庫不足時はエラー。

14. ECショップ

14.1 概要

Product モデルの `is_ec_visible=True` の商品が EC ショップに表示されます。

14.2 ページ構成

ページ	URL	説明
商品一覧	`/shop/`	EC 表示対象商品の一覧
カート	`/shop/cart/`	ショッピングカート
チェックアウト	`/shop/checkout/`	注文確定・決済

14.3 カート API

操作	API	メソッド
追加	`/api/shop/cart/add/`	POST
更新	`/api/shop/cart/update/`	POST
削除	`/api/shop/cart/remove/`	POST

14.4 多言語対応

`ProductTranslation` モデルで商品名・説明文を7言語に翻訳。ユーザーの言語設定に応じて自動切替。

15. セキュリティ機能

15.1 暗号化

対象	方式	キー環境変数
LINE User ID	Fernet 暗号化 + SHA-256 ハッシュ	`LINE_USER_ID_ENCRYPTION_KEY`, `LINE_USER_ID_HASH_PEPPER`
IoT Wi-Fi パスワード	Fernet 暗号化	`IOT_ENCRYPTION_KEY`
IoT API キー	SHA-256 ハッシュ（不可逆）	—
Email OTP	SHA-256 ハッシュ + 有効期限	—

15.2 セキュリティ監査

`python manage.py security_audit` または Celery タスク（毎日03:00）で自動実行。チェックカテゴリ：

- **django_settings** — DEBUG, ALLOWED_HOSTS, SECRET_KEY 等
- **credentials** — API キー・パスワードの安全性
- **endpoints** — エンドポイントのアクセス制御
- **infrastructure** — SSL, ファイアウォール
- **dependencies** — パッケージの脆弱性

15.3 監視ログ（SecurityLog）

`SecurityAuditMiddleware` が全リクエストを監視し、以下のイベントを自動記録：

event_type	severity	トリガー
login_success	info	ログイン成功
login_fail	warning	ログイン失敗
api_auth_fail	warning	/api/ への 401/403
permission_denied	warning	403 レスポンス
suspicious_request	critical	レート制限超過
admin_action	info	管理画面操作

15.4 レート制限

同一 IP から60秒間に100リクエスト超で `suspicious_request`（critical）を記録。カウンターは最大10,000エントリ。IP 検出は `X-Forwarded-For` ヘッダー対応。

15.5 本番セキュリティ設定

- `SECURE_SSL_REDIRECT = True`
- `SECURE_HSTS_SECONDS = 31536000`（1年）
- `SESSION_COOKIE_SECURE = True`
- `CSRF_COOKIE_SECURE = True`
- `X_FRAME_OPTIONS = "DENY"`

15.6 ログ保持

`cleanup_security_logs` タスク（毎週日曜04:00）で90日以上前のログを自動削除。`--days` オプションで保持期間変更可能。

16. AWSコスト監視

16.1 概要

`check_aws_costs` 管理コマンド（Celery で毎日06:00実行）が AWS リソースのコストを監視し、`CostReport` モデルに記録。

16.2 チェック対象

resource_type	チェック内容
ec2	EC2 インスタンスの稼働状況・コスト
s3	S3 バケットのストレージコスト
ebs	EBS ボリュームの使用状況
eip	未使用 Elastic IP
rds	RDS インスタンスのコスト
total	月額合計

16.3 ステータス

status	意味
ok	正常範囲内
warn	コスト上昇の兆候
alert	閾値超過（要対応）

16.4 使用方法

```
# 手動実行
python manage.py check_aws_costs

# 閾値指定
python manage.py check_aws_costs --threshold 100
```

17. 定期タスク（Celery）

17.1 Celery 設定

ブローカー: Redis（`CELERY_BROKER_URL`、デフォルト: `redis://localhost:6379/0`）

17.2 Beat スケジュール一覧

タスク名	関数	スケジュール	説明
delete-every-minute	delete_temporary_schedules	毎60秒	仮予約の自動削除（10分超過）
check-low-stock-daily	check_low_stock_and_notify	毎日 09:00	低在庫 LINE 通知
check-property-alerts	check_property_alerts	5分間隔	物件アラート監視
security-audit-daily	run_security_audit	毎日 03:00	セキュリティ監査
cleanup-security-logs-weekly	cleanup_security_logs	毎週日曜 04:00	古いログの削除
check-aws-costs-daily	check_aws_costs	毎日 06:00	AWS コスト監視

17.3 非同期タスク

タスク	トリガー	説明
trigger_gas_alert	IoT イベント受信時（mq9 超過）	ガス漏れ緊急アラート送信

17.4 起動方法

```
# ワーカー起動
celery -A project worker -l info

# Beat スケジューラー起動
celery -A project beat -l info

# 開発環境（ワーカー + Beat 同時）
celery -A project worker -l info -B
```

18. 管理コマンドリファレンス

コマンド	説明	主なオプション
`bootstrap_admin_staff`	管理者ユーザーと Staff レコード作成	`--username`, `--store_id`, `--manager`, `--developer`
`security_audit`	セキュリティ監査の実行	なし
`cleanup_security_logs`	古いセキュリティログの削除	`--days`（デフォルト: 90）
`check_aws_costs`	AWS コストチェック	`--threshold`
`cancel_expired_temp_bookings`	期限切れ仮予約のキャンセル	なし

18.1 使用例

```
# 管理者作成
python manage.py bootstrap_admin_staff \
  --username admin --store_id 1 --manager --developer

# セキュリティ監査
python manage.py security_audit

# 30日以上前のログ削除
python manage.py cleanup_security_logs --days 30

# AWS コストチェック（閾値 $50）
python manage.py check_aws_costs --threshold 50

# 期限切れ仮予約キャンセル
python manage.py cancel_expired_temp_bookings
```

19. デプロイ手順

19.1 サーバー前提条件

- OS: Ubuntu / Amazon Linux
- Python 3.x + venv
- Nginx, Gunicorn
- Redis（Celery ブローカー）
- PostgreSQL（RDS）— 本番は SQLite 使用不可
- SSL 証明書

19.2 本番デプロイ

```
# 1. リリースタグ作成
git tag -a release-YYYYMMDD-HHMM -m "Release description"
git push origin release-YYYYMMDD-HHMM

# 2. GitHub Actions が自動デプロイ
#    - deploy-production.yml が実行
#    - "DEPLOY" 確認文字列が必要
#    - セキュリティ監査・SECRET_KEY検証を事前実行

# 3. 手動デプロイの場合
ssh newfuhi@server
cd /var/www/newfuhi
git fetch && git checkout release-YYYYMMDD-HHMM
source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate --noinput
python manage.py collectstatic --noinput
sudo systemctl restart newfuhi-production
```

19.3 ステージングデプロイ

`develop` / `staging` ブランチへの push で自動デプロイ。rsync でコード同期後、マイグレーション・静的ファイル収集・サービス再起動。IP 制限あり（192.168.1.0/24, 10.0.0.0/8）。

19.4 設定ファイル

ファイル	説明
`config/nginx/production.conf`	Nginx 本番設定（HTTPS, TLS 1.2/1.3, HSTS）
`config/nginx/staging.conf`	Nginx ステージング設定（HTTP, IP制限, BasicAuth）
`config/gunicorn/production.conf.py`	Gunicorn 本番（workers=CPU*2+1, preload）
`config/systemd/newfuhi-production.service`	systemd サービス（セキュリティ制限付き）

19.5 CI/CD パイプライン

ワークフロー	トリガー	内容
pr-validation.yml	PR 作成	Lint（flake8, black, isort, mypy）、テスト、カバレッジ
deploy-staging.yml	develop/staging push	自動デプロイ + ヘルスチェック
deploy-production.yml	release-* タグ / 手動	承認必須デプロイ

19.6 ロールバック

```
ssh newfuhi@server
cd /var/www/newfuhi
git checkout release-PREVIOUS-TAG
source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate --noinput
sudo systemctl restart newfuhi-production
```

19.7 監視

```
# ログ確認
sudo journalctl -u newfuhi-production -f

# Nginx アクセスログ
tail -f /var/log/nginx/access.log

# ヘルスチェック
curl -s https://example.com/healthz
```

20. 多言語対応

20.1 対応言語

7言語対応。Django の i18n フレームワーク（`LocaleMiddleware`）+ URL プレフィックス（`i18n_patterns`）で言語切替。

20.2 翻訳ファイル

```
locale/
├── ja/LC_MESSAGES/django.po    # 日本語
├── en/LC_MESSAGES/django.po    # 英語
├── zh_Hant/LC_MESSAGES/django.po  # 繁體中文
├── zh_Hans/LC_MESSAGES/django.po  # 简体中文
├── ko/LC_MESSAGES/django.po    # 한국어
├── es/LC_MESSAGES/django.po    # Español
└── pt/LC_MESSAGES/django.po    # Português
```

20.3 翻訳コマンド

```
# メッセージ抽出
python manage.py makemessages -l ja
python manage.py makemessages -l en
# ... 全言語

# コンパイル
python manage.py compilemessages
```

20.4 商品翻訳（ProductTranslation）

商品名・説明文は `ProductTranslation` モデルで言語ごとに個別管理。管理画面からインラインで編集可能。翻訳がない言語では Product のデフォルト値（name, description）を表示。

21. トラブルシューティング

21.1 よくある問題

症状	原因	対処法
仮予約が消えない	Celery ワーカー / Beat が停止	`celery -A project worker -l info -B` で再起動
LINE ログインエラー	LINE_CHANNEL_ID / SECRET の設定ミス	環境変数を確認。LINE Developers コンソールでリダイレクト URL を確認
決済 Webhook が来ない	WEBHOOK_URL_BASE の設定ミス	本番 URL に設定されているか確認。ファイアウォール確認
IoT デバイスが接続できない	API キーの不一致	デバイス側のキーが登録時の平文と一致するか確認
メール送信失敗	SMTP 設定ミス	EMAIL_HOST, EMAIL_PORT, 認証情報を確認
静的ファイルが表示されない	collectstatic 未実行	`python manage.py collectstatic --noinput`
マイグレーションエラー	DB スキーマの不整合	`python manage.py showmigrations` で状態確認
管理画面のメニューが表示されない	Staff レコードなし / ロール未設定	`bootstrap_admin_staff` コマンドで作成
ガスアラートが発火しない	mq9_threshold 未設定 / alert_enabled=False	IoTDevice の閾値とアラート有効フラグを確認
給与計算が0円	EmploymentContract 未作成	スタッフの雇用契約を管理画面で設定

21.2 ログ確認方法

```
# Django アプリログ
tail -f debug.log

# 本番ログ（systemd）
sudo journalctl -u newfuhi-production -f --since "1 hour ago"

# Gunicorn ログ
tail -f /var/log/newfuhi/gunicorn-error.log

# Celery ログ
celery -A project worker -l debug

# セキュリティログ（DB）
python manage.py shell -c "
from booking.models import SecurityLog
for log in SecurityLog.objects.order_by('-created_at')[:10]:
    print(f'{log.created_at} [{log.severity}] {log.event_type}: {log.detail}')
"
```

21.3 緊急時の対応

状況	対応
サービス完全停止	`sudo systemctl restart newfuhi-production`
DB 接続エラー（本番）	RDS の状態確認、セキュリティグループ確認
ディスク容量不足	ログローテーション確認、`cleanup_security_logs` 実行
セキュリティインシデント	SecurityLog 確認、該当 IP ブロック、関係者に連絡
ロールバックが必要	前のリリースタグに checkout → サービス再起動